

MANIFOLD

Un motor matemático para mapear vulnerabilidades

Proyecto MANIFOLD

IR tipada · geometría · topología · verificación formal

2026

El problema: priorizar, no detectar

- Análisis estático basado en reglas y semántico: el cuello de botella real no es *detectar* sinks, sino *decidir cuál verificar*.
- En OWASP Benchmark 1.2 el índice de Youden $J = \text{TPR} - \text{FPR}$ colapsa: SonarQube $J \approx +0.01$, CodeQL $J \approx +0.22$.
- **Pregunta:** ¿puede un campo escalar $V(x)$ sobre el grafo de programa priorizar el sink vulnerable?

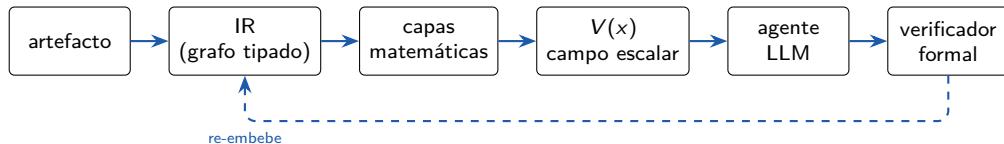
Experimento principal: oráculo de priorización

Ordenamiento (902 casos vulnerables)	MRR	R@1	NDCG@5
DFS (orden de fuente)	0.848	0.755	0.882
taint crudo	0.894	0.847	0.916
$V(x)$ fusionado	0.872	0.805	0.899
aleatorio	0.709 ± 0.042	0.531	0.778

AUC por señal (£predice el sink vulnerable?): taint 0.895 ∩ formal 0.878 ∩ espectral 0.496 ∩ topológica 0.500 ∩ geométrica **0.376**.

- $V(x)$ **no** supera al taint crudo: es ligeramente peor (0.872 vs. 0.894).
- Espectral/topológica: *indistinguibles del azar*; curvatura: *peor que el azar*.
- **Conclusión publicable (negativa)**: en OWASP Benchmark los priors geométrico y topológico *no sobreviven más allá del taint*.
- $\Rightarrow$ Valor restante: *witnesses formales*, el sustrato unificado y el diagnóstico de los cuellos de botella del análisis semántico. H1/H2/H3/H5: *no soportadas*.

La idea central



- Modelamos candidatos a vulnerabilidad como *violaciones de invariantes estructurales*, no como patrones puramente sintácticos.

Grafo de programa $G = (V, E, \tau_V, \tau_E, \omega)$:

- **Nodos**: módulo, función, sentencia, fuente, sumidero, gate, ...
- **Aristas tipadas**: control, data, call, taint, trust, auth
- JSON compartido Python $\leftrightarrow$ Rust; agnóstica al artefacto.

Multi-dominio: el *mismo* IR se produce desde

- Python / Java (tree-sitter), binarios (objdump o angr/CFGFast),
- OpenAPI (superficie de API), agentes LLM (grafo de herramientas/confianza).

Espectral ($L = D - A$)

λ_2 , vector de Fiedler, embedding. **Topológica**

(TDA)

homología persistente H_0, H_1 , Mapper.

Geométrica

curvatura de Ollivier–Ricci / Forman–Ricci;
Sinkhorn.

Algebraica

retículo de taint; orden de autorización.

Formal

ejecución simbólica + SMT (Z3);
alcanzabilidad. **Dirigida**

Laplaciano de Chung (SCC + Perron).

$$V(x) = \sigma(\sum_i \alpha_i s_i(x)) \quad (\text{pesos } \alpha_i \text{ calibrables})$$

Hipótesis de mapeo (rasgo $\rightarrow$ vulnerabilidad)

#	Rasgo matemático	Clase
H1	curvatura $\kappa \ll 0$ (cuello de botella)	escalada de privilegios
H2	generador de H_1 persistente	reentrancia / recursión
H3	corte de Fiedler	inyección cruzando confianza
H4	flujo de taint que cruza arista auth	violación de la invariante de autorización
H5	outlier de persistencia	rasgo real vs. espurio
H6	$\text{SAT}(\phi_{\text{malo}})$	witness (semántica codificada)

Son *hipótesis falsables*, no leyes; H1/H2/H3/H5 *no se sostienen* en los corpus evaluados (ver resultados).

El agente neuro-simbólico

- ➊ **Mapea:** IR + señales $V(x)$, persistencia, curvatura.
- ➋ **Hipotetiza:** el LLM propone sobre regiones priorizadas.
- ➌ **Verifica:** Z3 y chequeos semánticos confirman/refutan la hipótesis estructurada bajo el modelo.
- ➍ **Refina y reporta** (witness concreto).

Reparto de roles: la matemática propone *regiones*; el LLM propone *hipótesis*; el solver propone *pruebas*.

Gramática de verificación: spec estructurada (sink, fuente, línea, categoría) — el LLM nunca emite SMT-LIB.

Resultados: OWASP Benchmark 1.2

Herramienta	TPR	FPR	Prec.	F1	J	Tiempo
SonarQube ^(a)	0.956	0.946	0.330	0.490	+0.010	~3 min
CodeQL ^(a)	0.902	0.682	0.603	0.744	+0.220	~18 min
MANIFOLD adaptador ^(b)	0.842	0.674	0.572	0.681	+0.168	~45 s
MANIFOLD adaptador + Z3 ^(b)	0.834	0.776	0.549	0.662	+0.057	~4 min

(a) reportado en evaluaciones públicas; puede usar esquemas de agregación distintos — *no* es una comparación head-to-head hasta reejecutarlos bajo un protocolo uniforme. (b) **medido** (Z3 aporta witnesses, no precisión).

Ablación: de dónde viene la mejora

Variante (categorías de taint)	Prec.	Recall	F1
Adaptador naive inicial	0.515	0.426	0.466
+ branch-join, receptor/estado, cobertura	0.530	0.856	0.655
+ restricción XSS al writer (final)	0.549	0.834	0.662
final, sin branch-join	0.515	0.555	0.535

- **Branch-join** (supremo del retículo) es la palanca de *recall* (+0.28).
- La **restricción del writer HTTP** es la palanca de *precisión*.
- Los errores residuales *no* los recuperan *V* ni *Z3*: el campo empeora el ranking medio y el verificador *Z3* reproduce las métricas del adaptador. Los límites son estructurales (interprocedural) e internos a frameworks.

- **Mapa del manifold:** grafo IR con layout force-directed; nodos por tipo, aristas por tipo (taint rojo), gate morado.
- **Panel de hallazgos:** *confirmed* (Z3 con witness) vs *candidate*.
- **HTML autocontenido** (sin dependencias) + demo end-to-end sobre 10 artefactos.

```
python -m manifold.cli target -agent -viz out.html
```

Conclusiones y trabajo futuro

Conclusiones

- Una IR tipada enriquece señales independientes en un campo $V(x)$ navegable.
- El reparto neuro-simbólico (matemática/LLM/SMT) es el correcto: el LLM aporta interpretabilidad, no detección.
- **Resultado principal (negativo):** el dataflow semántico domina. El campo estructural ayuda solo con presupuestos muy pequeños ($P@10$) y perjudica en promedio (MRR 0.872 vs. 0.894; AP 0.434 vs. 0.489).
- La ingeniería guiada por datos funciona: el taint interprocedural subió el recall de Juliet 0.562 $\rightarrow$ 0.750; las summaries de frameworks capturan flujos a nivel aplicación.

Trabajo futuro

- Reejecución uniforme de CodeQL/SonarQube (caveat metodológico).
- Estudio de CVEs más amplio; corpus interprocedural/library-aware.
- Ablación mayor del LLM sobre OWASP (ranking/interpretación/reporte).

Gracias

Mapear el programa como un espacio — y probar si la estructura realmente ayuda.
Cuando la geometría falla, el fallo también es información.